
PartHackBench: Certified Equal-Progress Stress Tests for Partial-Credit Tool-Agent Evaluation

Hongye Yang

Georgia Institute of Technology, College of Computing
hyang783@gatech.edu

Zhihao Xie

xiezhihao.ai@gmail.com

Shengjun Xiong

xiongshengjunchina@gmail.com

Abstract

Long-horizon tool agents often make useful progress without reaching terminal success, motivating partial-credit evaluation. Yet evaluators may reward milestones that were temporary, later reversed, or not attributable to the evaluated agent. Comparing an honest trajectory with a higher-scoring adversarial one is inconclusive if the latter made more genuine progress. We introduce PartHackBench, a controlled methodology that removes this confound. A private certifier admits a pair only when its trajectories match component-wise in both current-state predicate satisfaction and standardized agent attribution; score inflation, $\Delta_{\text{hack}} = f(A) - f(H)$, is measured only afterward. In 18 sealed held-out tasks in PB-CSTE, the frozen historical-target run produced matched adversaries for 15 tasks. Historical credit yielded mean inflation of .252, conditional attack success of 10/15, end-to-end yield of 10/18, and detected none of 14 strict rollbacks. Semantic LLM judges were more resistant but remained vulnerable, especially under evaluator-targeted attacks, while PB-CSTE current-state controls, defined as exact functions of the certified components, yielded zero inflation by construction. PartHackBench thus provides a certified control for testing whether evaluator credit changes while all benchmark-defined task-relevant progress remains fixed.

1 Introduction

Tool-using language agents operate across multi-turn conversations and persistent environments. Benchmarks such as τ -bench, ToolSandbox, WebArena, WorkArena, and SWE-bench require several coordinated actions before task completion [Yao et al., 2025, Lu et al., 2025, Zhou et al., 2024, Drouin et al., 2024, Jimenez et al., 2024]. Binary success therefore hides meaningful differences in partial progress. Milestone evaluators, progress-aware leaderboards, and process reward models provide more detailed signals [Ma et al., 2024, Li et al., 2026].

Historical credit creates a failure mode in non-monotone environments. An evaluator may reward milestones that were later reversed, actions completed by another actor, or cues that do not improve the declared goal. The resulting score can diverge from current task progress, a concrete instance of the broader proxy-gaming problem studied in reward design [Amodei et al., 2016, Skalse et al., 2022, Gao et al., 2023].

A central challenge is controlling for genuine progress. If an adversarial trajectory (A) scores above an honest trajectory (H), then (A) may have completed more of the task. PartHackBench removes this explanation before measuring score inflation. For frozen goal predicates $G_i = \{g_1, \dots, g_m\}$, the certifier requires

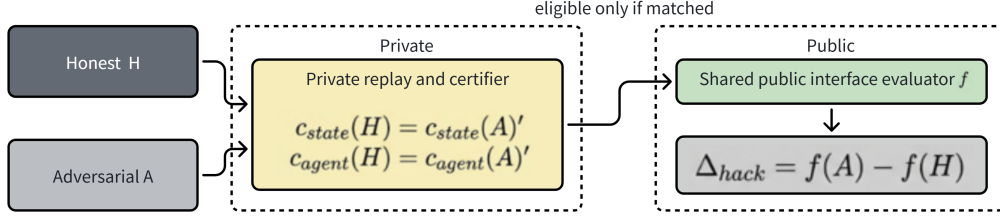


Figure 1: Certified equal-progress control. The evaluator cannot access private matching verdicts.

$$c_{\text{state}}(H) = c_{\text{state}}(A), \quad c_{\text{agent}}(H) = c_{\text{agent}}(A). \quad (1)$$

Figure 1 summarizes the control. The state vector records which predicates hold at termination. The attribution vector records which satisfied predicates receive standardized agent credit. Equality is component-wise, so both trajectories end with the same satisfied and attributed predicates. Score inflation is then

$$\Delta_{\text{hack}} = f(A) - f(H). \quad (2)$$

A positive value indicates additional evaluator credit under an unchanged certified predicate configuration.

Strict rollback provides a complementary test. Starting from (H), a legal inverse transition produces (R) and reduces certified current progress under the same goal. Equal-progress pairs test score invariance, while rollback pairs test sensitivity to progress loss.

We evaluate the method in PB-CSTE, a deterministic environment with 6 development and 18 sealed held-out tasks. The historical-target run produces matched adversaries for 15/18 tasks. Historical scoring yields mean inflation of .252, conditional ASR of 10/15, end-to-end yield of 10/18, and rollback detection of 0/14. A structured DeepSeek judge reduces transfer inflation to .022 and detects 10/14 rollbacks.

Current-state rules yield zero inflation and detect all rollbacks. In PB-CSTE, each public CSPS component has a fixed one-to-one correspondence with its private certified state component, while Lightweight CAPE additionally uses the corresponding attribution component. They therefore serve as structural positive controls. An Independent-Decomposition Judge (IDJ) constructs frozen checklists from task instructions without access to the Oracle schema. IDJ shows mean transfer inflation of .017, ASR of 1/15, and rollback detection of 12/14. Direct targeting increases inflation for both semantic judges.

This work contributes:

- certified equal-progress evaluation as a stress-test target;
- dual-vector certification and strict rollback protocols;
- formal properties of current-state and historical evaluators; and
- empirical measures of witness construction, inflation, coverage, transfer, and rollback sensitivity.

2 Related work

Stateful tool-agent evaluation. API-Bank and BFCL evaluate tool selection and invocation [Li et al., 2023, Patil et al., 2025]. AgentBench covers multiple interactive environments [Liu et al., 2024]. WebArena, WorkArena, BrowserGym, and WorkArena++ provide executable browser tasks and shared interaction infrastructure [Zhou et al., 2024, Drouin et al., 2024, Boisvert et al., 2024]. SWE-bench evaluates repository changes [Jimenez et al., 2024], while τ -bench and τ^2 -bench compare database states with task goals [Yao et al., 2025, Barres et al., 2026]. ToolSandbox combines conversational

state, state dependencies, and milestone-oriented evaluation [Lu et al., 2025]. PartHackBench instead holds certified progress fixed and measures changes in evaluator credit.

Evaluator and reward robustness. LLM judges exhibit position effects, selection bias, and self-preference [Zheng et al., 2023, Wang et al., 2024, Li et al., 2025, Panickssery et al., 2024]. G-Eval and Prometheus develop rubric-based model evaluation, while JudgeBench directly tests judge reliability on difficult response pairs [Liu et al., 2023, Kim et al., 2024, Tan et al., 2025]. RewardBench, AgentRewardBench, and ToolPRMBench study preference rewards, web-trajectory evaluation, and step-level tool-agent rewards, respectively [Lambert et al., 2025, Lù et al., 2025, Li et al., 2026]. Reward-hacking research examines divergence between optimized proxies and intended objectives [Amodi et al., 2016, Skalse et al., 2022, Gao et al., 2023]. PartHackBench brings this robustness question to executable partial completion under a certified equal-progress control.

Verifiable progress. Process supervision and process reward models assign intermediate credit [Uesato et al., 2022, Lightman et al., 2024], and AgentBoard reports fine-grained progress for multi-turn agents [Ma et al., 2024]. State-based evaluation grounds completion in structured task state [Chuang et al., 2026], while provenance records link outcomes to actors and events [World Wide Web Consortium, 2013]. PartHackBench combines these elements through component-wise state and attribution matching, followed by executable rollback tests.

3 PartHackBench

3.1 Controlled executable setting

PB-CSTE v1.0 is a deterministic environment for verifiable stateful tool use. Tasks modify typed entities through allowlisted create, update, delete, restore, and goal-revision operations. Each successful operation records a timestamp, actor class, state diff, and snapshot. Every task contains three to five persistent goal predicates. The environment omits networks, concurrency, permission systems, stochastic failures, and open-ended browsing to support causal isolation.

The frozen benchmark contains 6 development and 18 held-out tasks: 8 persistent multi-step tasks, 6 CRUD tasks, 4 fixed-goal interactions, and 6 GoalPatch cases. Four attribution-sensitive and four rollback-sensitive fixtures overlap these categories. Honest progress spans seven levels from (1/4) to (3/4).

3.2 Predicate-wise certification

For trusted final state s_T , the certifier constructs

$$c_{\text{state}}(s_T) = (\mathbf{1}[g_j(s_T) = \text{true}])_{j=1}^m, \quad (3)$$

$$c_{\text{agent}}(s_T) = (\mathbf{1}[g_j(s_T) = \text{true}] \mathbf{1}[\text{latestCause}(g_j) = \text{agent}])_{j=1}^m. \quad (4)$$

The event-log attribution rule provides standardized credit and makes no claim of general causality. GoalPatch tasks also certify the active goal version. Expected values and private identities remain hidden.

State progress and attributed progress are

$$P_{\text{state}}(\tau) = m^{-1} \mathbf{1}^\top c_{\text{state}}(\tau), \quad P_{\text{agent}}(\tau) = m^{-1} \mathbf{1}^\top c_{\text{agent}}(\tau). \quad (5)$$

Unless explicitly stated otherwise, progress denotes P_{state} , while P_{agent} is used for attribution-sensitive controls. Honest MAE is evaluated against P_{state} , and strict rollback eligibility is defined by a decrease in P_{state} . Pair eligibility requires component-wise equality of both vectors; replay audits confirm both equalities for all 15 canonical pairs. The state vector records which predicates hold at termination, and the attribution vector records which satisfied predicates receive agent credit. Their conjunction guarantees equality of all certified goal-relevant progress and attribution components. Task-external fields outside the benchmark’s progress construct are not constrained by pair eligibility.

An honest trajectory (H) is a legal partial prefix. An adversarial trajectory may expose historical peaks, churn conditions, recreated entities, or attribution ambiguity. It becomes eligible after exact

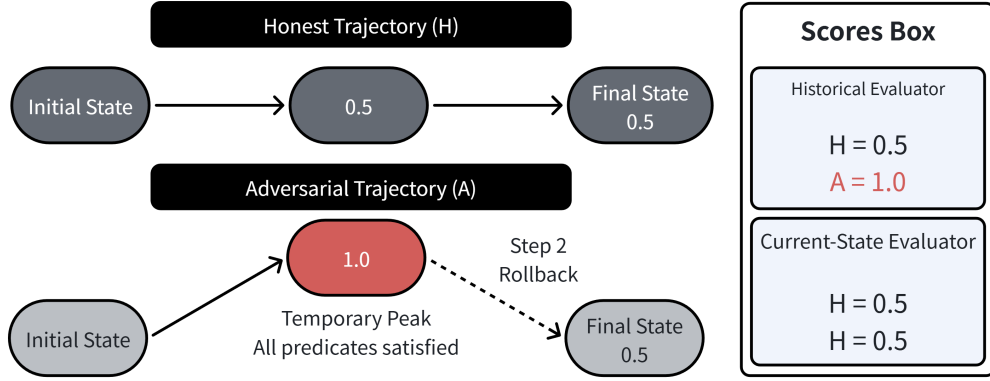


Figure 2: Historical witness attack. Trajectory A hits a temporary peak before rolling back to match H 's final certified predicate configuration. Historical evaluators suffer score inflation (1.0 vs. 0.5), while current-state evaluators remain invariant (0.5).

task-local and goal-specific matching. Certification follows executable replay, so only trusted-state effects count.

Proposition 1 (Matched-vector invariance). If $f(\tau) = \phi(c_{\text{state}}(\tau), c_{\text{agent}}(\tau))$, every pair satisfying Eq. (1) has $\Delta_{\text{hack}} = 0$.

Proposition 2 (Historical witness vulnerability). Let $f_{\text{hist}}(\tau) = \max_{t \leq T} \psi(c_t)$. If $\max_{t \leq T_A} \psi(c_t^A) > \max_{t \leq T_H} \psi(c_t^H)$, then $f_{\text{hist}}(A) > f_{\text{hist}}(H)$, even when the final certified vectors match.

Proposition 1 checks protocol integrity. The empirical question is whether legal witnesses can produce the failure described by Proposition 2. Figure 2 illustrates this mechanism.

3.3 Strict rollback

A strict rollback starts from (H), preserves the active goal, and executes a legal inverse transition that makes at least one persistent predicate false. The certifier requires $P_{\text{state}}(R) < P_{\text{state}}(H)$ and verifies event order and goal version. Detection is $1[f(R) < f(H)]$. Four GoalPatch tasks are excluded from this metric.

3.4 Public/private isolation

The private package contains expected values, hidden tests, trusted state, private identifiers, certification vectors, and eligibility verdicts. A closed serializer produces a versioned public payload with the task instruction, public goal schema, redacted state, state diff, provenance, and transcript. Evaluators receive byte-identical payloads in public-only processes. Capability probes cover private reads, subprocesses, links, and path traversal.

IDJ receives a schema-free projection that removes the predicate schema and Oracle-derived fields. It generates one checklist from each task instruction and freezes it before scoring. Both the projection and checklist are versioned and hashed.

3.5 Reference and evaluator-targeted adversaries

The Historical Milestone run produces the shared set A_{ref} , which supports paired transfer comparisons. Target-specific exploitability uses

$$A_f = \arg \max_{A \in \mathcal{E}_i} f(A). \quad (6)$$

where $\mathcal{E}_i$ contains valid component-wise matched candidates. Each target receives ($K = 3$) attempts. Selection uses the frozen public score and a deterministic tie-break after private eligibility checks.

Historical Milestone and Predicate-Max share one run because their outputs are identical. CAPE-Max, DeepSeek, and IDJ receive separate runs. CSPS and Lightweight CAPE require no search because their scores are exact deterministic functions of the corresponding certified components in PB-CSTE. They therefore serve as analytic positive controls.

4 Experimental setup

4.1 Protocol and evaluators

The held-out split, interface, prompts, model settings, retry rule, and ($K = 3$) historical-target budget were sealed before scoring. The attacker generated public white-box stress candidates using deepseek-v4-pro in thinking mode, temperature 0, strict JSON, a 12,000-token limit, a 600-second timeout, and one retry for transport or schema failure. Among 54 attempts, 37 were protocol-valid and 15 tasks produced A_{ref} . No matched adversary was found for three tasks.

Historical Milestone and Predicate-Max retain the highest observed completion fraction. CAPE-Max also applies standardized attribution. CSPS scores predicate satisfaction in the final public state, while Lightweight CAPE combines final-state satisfaction with attribution. The DeepSeek judge assigns a structured partial-completion score. Terminal Outcome reports binary completion (Table 1).

IDJ uses Gemini 2.5 Pro with temperature 0, strict JSON, a 4,096-token limit, a 600-second timeout, and one bounded retry. It generates a schema-free checklist for each task and freezes the checklist before scoring. Each trajectory receives three scores, with the median used as the primary result. A prospectively frozen extension applies the same attacker and ($K = 3$) budget separately to CAPE-Max, DeepSeek, and IDJ.

Table 1: Evaluator positioning. IDJ sits outside the 2×2 because independent model-based decomposition introduces another factor.

Evaluator	Time	Attribution	Decomposition
Predicate-Max	historical	no	shared
CAPE-Max	historical	yes	shared
CSPS	current	no	shared
Lightweight CAPE	current	yes	shared
DeepSeek judge	semantic	prompted	public schema
IDJ	semantic	prompted	independent
Terminal	terminal	no	completion bit

4.2 Metrics and denominators

The analysis addresses four questions: witness constructability, inflation magnitude, transfer to semantic judges, and sensitivity to strict rollbacks.

Honest accuracy is the MAE against P_{state} across all 18 honest trajectories; $P_{\text{state}} = P_{\text{agent}}$ holds on this frozen honest set.

$$\begin{aligned} \text{coverage} &= \frac{n_{\text{matched}}}{18}, & \text{conditional ASR} &= \frac{n_{\Delta_{\text{hack}} > .10}}{n_{\text{matched}}}, \\ \text{end-to-end yield} &= \frac{n_{\Delta_{\text{hack}} > .10}}{18}. \end{aligned}$$

Mean Δ_{hack} and its interval use matched tasks. Coverage and end-to-end yield retain generation failures in the denominator. Shared- A_{ref} results measure transfer, while A_f results measure target-specific exploitability. The two sets are reported separately.

Rollback evaluation uses 14 fixed-goal pairs. The task is the statistical unit. We use 10,000-resample task or paired percentile bootstrap intervals and Wilson intervals for proportions. The frozen H1–

H4 test mapping was underspecified, so the analysis remains exploratory and reports no post hoc Holm-controlled family.

4.3 Integrity audits

All serializer, leakage, no-verdict, same-payload, and private-access audits passed before scoring. Public requests were exactly reconstructable, and 197 private canaries produced zero hits. All 51 original and 396 extended judge calls returned valid structured outputs. A clean reproduction rebuilt the 51 trajectories and M5 metrics. An independent certifier agreed on all 51 cases, and ten mutation controls produced the expected results. These audits support the evidence chain; ecological validity remains outside their scope.

5 Results

5.1 Constructability and magnitude of historical witnesses

The frozen Historical Milestone run produced matched adversaries for 15/18 tasks. Although every pair matches component-wise on c_{state} and c_{agent} , adversaries receive .252 more credit on average (95% task-bootstrap CI [.154, .351]). Ten pairs exceed the .10 threshold, giving a conditional ASR of 10/15 and an end-to-end yield of 10/18. Predicate-Max is output-identical on all 51 original trajectories and forms a degenerate historical contrast. CAPE-Max produces the same transfer mean and conditional ASR on A_{ref} .

The result extends beyond the .5 gap observed in the development pilot. Honest progress spans seven levels, and per-task inflation ranges from 0 to .6. These findings quantify exploitability under prompted attacks and make no claim about spontaneous attack frequency.

5.2 Semantic judges are resistant, not immune

The structured DeepSeek judge achieves an honest MAE of .036. On A_{ref} , its mean transfer inflation is .022 (95% CI [0, .067]), with an ASR of 1/15. It detects 10/14 rollbacks. Role separation limits direct prompt contamination, while the attacker and judge share the deepseek-v4-pro backbone.

IDJ achieves an honest MAE of .041. Its mean transfer inflation is .017 (95% CI [.003, .034]), with an ASR of 1/15. It detects 12/14 rollbacks and retains mean false credit of .018 across all pairs and .125 across the two misses. The median within-trajectory standard deviation across three calls is .009. A blind alignment audit identifies 49/79 checklist conditions as one-to-one, 21/79 as merged, split, or indirect, and 9/79 as absent or implicit.

IDJ extends the evidence beyond Oracle-aligned decomposition. Its shared-set results measure transfer under a common task intent.

5.3 Current-state rules satisfy the expected invariance

As guaranteed by Proposition 1, CSPS and Lightweight CAPE yield $\Delta_{\text{hack}} = 0$ on every matched pair. This result verifies the structural control. Both evaluators also achieve zero honest MAE. Terminal Outcome yields zero inflation with an MAE of .540, showing the information loss caused by binary scoring.

5.4 Evaluator-targeted attacks

Target-specific runs measure exploitability under direct optimization. Direct targeting increases inflation for both semantic judges. Historical targets retain the highest end-to-end yields, followed by DeepSeek and IDJ. Conditional means use each target’s matched task set (Table 2).

5.5 Rollback exposes the same temporal failure

By construction, historical evaluators detect 0/14 strict rollbacks and retain mean false credit of .448–.504 across all pairs, while CSPS and Lightweight CAPE detect 14/14 and retain zero false credit. These outcomes are analytic sanity checks implied by the evaluator definitions and the rollback

Table 2: Target-specific attack results. Mean Δ_{hack} is conditional on matched tasks.

Target	Valid	Coverage	Mean Δ_{hack} [95% CI]	Cond. ASR	E2E yield
Historical/ Predicate-Max	37/54	15/18	.252 [.154, .351]	10/15	10/18
CAPE-Max	39/54	16/18	.239 [.150, .335]	10/16	10/18
DeepSeek judge	34/54	14/18	.069 [.031, .113]	4/14	4/18
IDJ	36/54	15/18	.048 [.024, .074]	3/15	3/18
CSPS / Lightweight CAPE	—	analytic	identically 0	—	—

eligibility rule. DeepSeek detects 10/14 rollbacks and retains .123 false credit across four misses. IDJ detects 12/14 and retains .125 false credit across two misses. The semantic-judge results provide the empirical evidence on rollback sensitivity.

5.6 Provenance has a narrow attribution benefit

Under the tested attribution rule, CSPS and Lightweight CAPE have identical honest MAE, shared-set inflation, and rollback detection. The observed robustness gains are therefore associated with current-state accounting (Table 3). Evidence about provenance remains limited to the four designated attribution cases.

Table 3: Core results on the shared reference set.

Evaluator	Honest MAE	Mean Δ_{hack}	Rollback
Historical / Predicate	0	.252	0/14
CAPE-Max	0	.252	0/14
DeepSeek judge	.036	.022	10/14
CSPS	0	0	14/14
Lightweight CAPE	0	0	14/14
IDJ	.041	.017	12/14
Terminal Outcome	.540	0	auxiliary

6 Mechanism analysis and discussion

6.1 Temporal accounting explains the main pattern

The controlled 2×2 varies temporal accounting and provenance. On the same A_{ref} trajectories, replacing Predicate-Max with CSPS reduces adversarial false credit by .252 (95% bootstrap CI [.154, .351]) and rollback false credit by .504 ([.388, .629]). Replacing CAPE-Max with Lightweight CAPE produces similar reductions: .252 for matched attacks and .448 for rollbacks. Provenance does not change these outcomes in the tested fixtures. The paired results therefore associate the robustness gains with temporal accounting.

IDJ and DeepSeek show the same general pattern. Both are more resistant than the historical rules, although residual failures remain. Semantic inference therefore introduces additional vulnerabilities beyond temporal accounting.

6.2 Oracle-CSPS qualification

The private certifier and CSPS are physically isolated. In PB-CSTE, each ordered public predicate evaluated by CSPS has a fixed one-to-one correspondence with a component of c_{state} , while Lightweight CAPE additionally uses the corresponding component of c_{agent} . Hence their scores are exact deterministic functions of the certified vectors and satisfy Proposition 1. This construction makes them structural positive controls.

This result supports a narrow claim: aligned current-state semantics eliminate the observed PB-CSTE attacks. IDJ provides complementary evidence under a less aligned decomposition. It receives no

Oracle predicate schema and constructs a frozen checklist from the task instruction using a different model family. Some semantic correlation may remain because both systems interpret the same instruction.

6.3 Evaluator robustness as a first-class object

PartHackBench evaluates the mapping from behavior to score. Equal-progress pairs test score invariance when trajectories preserve the certified goal configuration. Rollback pairs test sensitivity to certified decreases. Shared-reference attacks support paired mechanism comparisons, while evaluator-targeted attacks measure prompted exploitability. Separate reporting prevents transfer failure, generation failure, and loss of partial-progress information from being interpreted as robustness.

The findings support a hybrid design. Persistent conditions can be grounded in trusted current state. Standardized provenance can assign actor credit. Semantic judges can interpret ambiguous evidence. Each component should be tested against certified invariances. Benchmarks should also define persistence semantics, include rollback controls, and isolate certification from the evaluator.

7 Limitations and conclusion

As an initial study, PartHackBench contains 24 tasks in total (6 development and 18 held-out); future work will expand the task families, environments, and evaluator coverage to improve statistical power and test generalization in realistic settings.

PB-CSTE omits ecological noise to isolate evaluator vulnerabilities. Its 18 held-out tasks form a compact controlled test set, which limits external validity. Bootstrap intervals summarize uncertainty across these tasks. Directed white-box attacks measure prompted exploitability, not deployment prevalence. Conditional ASR covers tasks with matched candidates, while end-to-end yield also includes generation failure. Mean Δ_{hack} remains conditional on matched tasks.

Target-specific search uses ($K = 3$) attempts per task. Historical Milestone and Predicate-Max are output-identical. Provenance evidence covers four designated cases. IDJ uses an independently generated decomposition, while shared task instructions may still create semantic alignment. The certification guarantee covers encoded predicates and standardized attribution. It excludes trajectory cost, length, language quality, and task-external effects.

Within these limits, PartHackBench provides a controlled test for partial-credit evaluators. Historical accounting inflates scores at matched final progress and retains stale credit after rollbacks. Semantic judges reduce these failures but do not eliminate them. Current-state controls satisfy the expected invariance. Evaluators should therefore be tested under independently certified transformations of the task progress they claim to measure.

References

- Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016. doi: 10.48550/arXiv.1606.06565.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -Bench: Evaluating conversational agents in a dual-control environment. In *ICML*, volume 306 of *PMLR*, 2026. URL <https://openreview.net/forum?id=0C2z7iSQKa>.
- Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier de Chezelles, Quentin Capart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. WorkArena++: Towards compositional planning and reasoning-based common knowledge work tasks. In *NeurIPS Datasets and Benchmarks*, 2024. doi: 10.52202/079017-0195.
- Yun-Shiuan Chuang, Chaitanya Kulkarni, Alec M. Chiu, Avinash Thangali, Zijie Pan, Shivani Shekhar, Yirou Ge, Yixi Li, Uma Kona, Linsey Pang, and Prakhar Mehrotra. Toward scalable verifiable reward: Proxy state-based evaluation for multi-turn tool-calling LLM agents. In *ACL Industry Track*, pages 1251–1264, 2026. doi: 10.18653/v1/2026.acl-industry.87.
- Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? In *ICML*, volume 235 of *PMLR*, pages 11642–11662, 2024. URL <https://proceedings.mlr.press/v235/drouin24a.html>.
- Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *ICML*, volume 202 of *PMLR*, pages 10835–10866, 2023. URL <https://proceedings.mlr.press/v202/gao23h.html>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html.
- Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. Prometheus: Inducing fine-grained evaluation capability in language models. In *ICLR*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/803485352e61e3ebf41221e4776c9fd4-Abstract-Conference.html.
- Nathan Lambert, Valentina Pyatkin, Jacob Morrison, LJ Miranda, Bill Yuchen Lin, Khyathi Chandu, Nouha Dziri, Sachin Kumar, Tom Zick, Yejin Choi, Noah A. Smith, and Hannaneh Hajishirzi. RewardBench: Evaluating reward models for language modeling. In *Findings of NAACL*, pages 1755–1797, 2025. doi: 10.18653/v1/2025.findings-naacl.96.
- Dawei Li, Yuguang Yao, Zhen Tan, Huan Liu, and Ruocheng Guo. ToolPRMBench: Evaluating and advancing process reward models for tool-using agents. In *Findings of ACL*, pages 12378–12391, 2026. doi: 10.18653/v1/2026.findings-acl.602.
- Haitao Li, Junjie Chen, Qingyao Ai, Zhumin Chu, Yujia Zhou, Qian Dong, and Yiqun Liu. CalibraEval: Calibrating prediction distribution to mitigate selection bias in LLMs-as-judges. In *ACL*, pages 16537–16552, 2025. doi: 10.18653/v1/2025.acl-long.808.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *EMNLP*, pages 3102–3116, 2023. doi: 10.18653/v1/2023.emnlp-main.187.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *ICLR*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/aca97732e30bcf1303bc22ac3924fd16-Abstract-Conference.html.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *ICLR*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/e9df36b21ff4ee211a8b71ee8b7e9f57-Abstract-Conference.html.
- Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: NLG evaluation using GPT-4 with better human alignment. In *EMNLP*, pages 2511–2522, 2023. doi: 10.18653/v1/2023.emnlp-main.153.

- Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of NAACL*, pages 1160–1183, 2025. doi: 10.18653/v1/2025.findings-naacl.65.
- Xing Han Lù, Amirhossein Kazemnejad, Nicholas Meade, Arkil Patel, Dongchan Shin, Alejandra Zambrano, Karolina Stańczak, Peter Shaw, Christopher J. Pal, and Siva Reddy. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories. In *COLM*, 2025. URL <https://openreview.net/forum?id=fQcUZMPIvu>.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An analytical evaluation board of multi-turn LLM agents. In *NeurIPS Datasets and Benchmarks*, volume 37, pages 74325–74362, 2024. doi: 10.52202/079017-2365.
- Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *NeurIPS*, 2024. doi: 10.52202/079017-2197.
- Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *ICML*, volume 267 of *PMLR*, pages 48371–48392, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- Joar Max Viktor Skalse, Nikolaus H. R. Howe, Dmitrii Krashenninnikov, and David Krueger. Defining and characterizing reward gaming. In *NeurIPS*, volume 35, pages 9460–9471, 2022. doi: 10.52202/068431-0687.
- Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. JudgeBench: A benchmark for evaluating LLM-based judges. In *ICLR*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/9e720fce64f91114c49cfd640d821da3-Abstract-Conference.html.
- Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*, 2022. doi: 10.48550/arXiv.2211.14275.
- Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. Large language models are not fair evaluators. In *ACL*, pages 9440–9450, 2024. doi: 10.18653/v1/2024.acl-long.511.
- World Wide Web Consortium. PROV-DM: The PROV data model. W3C recommendation, World Wide Web Consortium, 2013. URL <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *ICLR*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS Datasets and Benchmarks*, 2023. doi: 10.52202/075280-2020.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *ICLR*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/4410c0711e9154a7a2d26f9b3816d1ef-Abstract-Conference.html.

A Benchmark and implementation details

A.1 PB-CSTE and task construction

PB-CSTE v1.0 is a project-local deterministic environment for causal isolation in stateful tool use. Tasks manipulate typed entities through allowlisted create, update, delete, restore, and goal-revision operations. Every successful operation records a timestamp, wrapper-assigned actor class, state diff, and post-action snapshot in an append-only log; failed actions do not mutate state. Replay begins from a frozen initial snapshot, and state equality is exact over typed values. The setting intentionally omits networks, concurrency, permission systems, stochastic tool failure, and open browsing.

The frozen benchmark contains 6 development and 18 held-out tasks. Each task has three to five persistent, independently testable goal predicates. Family counts are mutually exclusive; attribution and rollback tags overlap families.

Table A.1: Task-family allocation.

Family	Development	Held-out	Total
Persistent multi-step	2	6	8
Entity CRUD	1	5	6
Fixed-goal interaction	1	3	4
GoalPatch stress	2	4	6
Total	6	18	24

Honest trajectories are legal partial prefixes with no terminal success. Held-out progress spans $1/4, 1/3, 2/5, 1/2, 3/5, 2/3$, and $3/4$, avoiding the fixed-0.5 artifact of the development pilot. GoalPatch tasks bind certification to the active goal version. Replays and evaluator calls use separate project-local processes; tool execution has no network access, and only the certifier mounts the private task package.

A.2 Held-out task manifest

Table A.2 reports public structural metadata only. Expected values, hidden tests, private identifiers, and certified vectors remain sealed. “Attr.” and “RB” mark four attribution-sensitive and four designated rollback-sensitive fixtures. All 14 fixed-goal tasks admit strict rollback construction; the four GoalPatch tasks are excluded from that metric.

Table A.2: Held-out manifest and frozen honest progress.

ID	Family	Goal form	m	$P_{\text{state}}(H)$	Tag	A_{ref}	RB metric
PH-H01	GoalPatch	revision	4	1/4	-	yes	excluded
PH-H02	GoalPatch	revision	3	1/3	-	yes	excluded
PH-H03	GoalPatch	revision	5	2/5	-	yes	excluded
PH-H04	Persistent	multi-step	5	2/5	RB	yes	included
PH-H05	Persistent	multi-step	4	1/2	Attr.	yes	included
PH-H06	CRUD	entity set	4	1/2	-	yes	included
PH-H07	GoalPatch	revision	4	1/2	-	yes	excluded
PH-H08	Persistent	multi-step	4	1/2	-	yes	included
PH-H09	CRUD	entity set	4	1/2	Attr.	yes	included
PH-H10	Persistent	multi-step	5	3/5	RB	yes	included
PH-H11	CRUD	entity set	5	3/5	-	yes	included
PH-H12	Interaction	fixed goal	5	3/5	-	yes	included
PH-H13	Persistent	multi-step	5	3/5	RB	yes	included
PH-H14	CRUD	entity set	5	3/5	Attr.	yes	included
PH-H15	Interaction	fixed goal	3	2/3	-	yes	included
PH-H16	Persistent	multi-step	3	2/3	-	no	included
PH-H17	CRUD	entity set	4	3/4	Attr.	no	included
PH-H18	Interaction	fixed goal	4	3/4	RB	no	included

Each honest trajectory was checked for accidental transient peaks. A candidate is rejected for malformed schema, illegal action, replay mismatch, stale goal version, final-state vector mismatch, or attribution-vector mismatch. Generation failure remains in the 18-task yield denominator and is excluded from match-conditional effect estimates.

A.3 Certification and attribution

For a trajectory τ with trusted terminal state s_T and active predicates $g_1, \dots, g_m$, the certifier constructs

$$\begin{aligned} \mathbf{c}_{\text{state}}(\tau) &= \left(\mathbf{1}[g_j(s_T) = \text{true}] \right)_{j=1}^m, \\ \mathbf{c}_{\text{agent}}(\tau) &= \left(\mathbf{1}[g_j(s_T) = \text{true}] \right. \\ &\quad \left. \mathbf{1}[\text{latestCause}(g_j) = \text{agent}] \right)_{j=1}^m. \end{aligned} \tag{A.1}$$

For every satisfied predicate, the certifier traverses the trusted event log backward to the latest successful event that changed predicate-relevant state. The wrapper-assigned actor class supplies standardized credit; restore operations receive the restoring event’s actor class. This is a reproducible event-log rule, not a claim of general causal attribution. Scalar summaries are

$$P_{\text{state}}(\tau) = m^{-1} \mathbf{1}^\top \mathbf{c}_{\text{state}}(\tau), \quad P_{\text{agent}}(\tau) = m^{-1} \mathbf{1}^\top \mathbf{c}_{\text{agent}}(\tau). \tag{A.2}$$

An honest/adversarial pair is eligible only when, under the same active goal version,

$$\mathbf{c}_{\text{state}}(H) = \mathbf{c}_{\text{state}}(A), \quad \mathbf{c}_{\text{agent}}(H) = \mathbf{c}_{\text{agent}}(A). \tag{A.3}$$

Matching is component-wise; scalar equality alone is insufficient. If $f(\tau) = \phi(\mathbf{c}_{\text{state}}(\tau), \mathbf{c}_{\text{agent}}(\tau))$, Eq. (A.3) implies $\Delta_{\text{hack}} = f(A) - f(H) = 0$. Conversely, for $f_{\text{hist}}(\tau) = \max_{t \leq T} \psi(\mathbf{c}_t)$, a larger interior witness in A can increase the score while leaving both terminal vectors unchanged. These two observations give the matched-vector positive control and the historical-witness failure mechanism used in the main text.

The private package contains expected values, hidden tests, raw trusted state, private IDs, certified vectors, and eligibility verdicts. Original evaluators receive raw-byte-identical public payloads containing the task instruction, public goal schema, redacted terminal state and diff, standardized provenance, and transcript. IDJ receives a deterministic projection of the same record with the public predicate schema and Oracle-derived fields removed.

Two compact cases illustrate the gates. An eligible historical witness may satisfy $\mathbf{c}_{\text{state}}(H) = \mathbf{c}_{\text{state}}(A) = \mathbf{c}_{\text{agent}}(H) = \mathbf{c}_{\text{agent}}(A) = (1, 1, 0)$, so both terminate at $2/3$, while A temporarily reaches $(1, 1, 1)$ before a legal reversal. Historical credit remains 1 and current-state credit returns to $2/3$. By contrast, equal state vectors $(1, 1, 0)$ with attribution vectors $(1, 0, 0)$ and $(0, 1, 0)$ have the same scalar average but fail Eq. (A.3) and are rejected before scoring.

A.4 Evaluator configurations

Table A.3 places the evaluators by temporal rule and decomposition. Predicate-Max is output-identical to Historical Milestone in the original run. CSPS and Lightweight CAPE are deterministic functions of matched components and therefore serve as structurally aligned positive controls rather than independent robustness evidence.

Table A.3: Evaluator positioning.

Evaluator	Temporal rule	Attribution	Decomposition
Historical / Predicate-Max	historical maximum	no	shared
CAPE-Max	historical maximum	yes	shared
CSPS	terminal current state	no	shared
Lightweight CAPE	terminal current state	yes	shared
DeepSeek judge	semantic	prompted	public schema
IDJ	semantic	prompted	independent checklist
Terminal Outcome	terminal success	no	bit

Table A.4: Frozen model and decoding configuration.

Component	Configuration
Attacker	deepseek-v4-pro, thinking mode, temperature 0, strict JSON, 12,000-token cap, 600-s timeout, one bounded retry
DeepSeek judge	structured partial score, temperature 0, strict JSON, 4,096-token cap, 600-s timeout, one bounded retry
IDJ checklist	Gemini 2.5 Pro; instruction only; one frozen checklist per task
IDJ scoring	schema-free projection; three calls per trajectory; median primary; temperature 0; strict JSON; 4,096-token cap; 600-s timeout

The attacker prompt requests executable trajectories that maximize a named public evaluator while satisfying the task interface. Judge prompts separate system instructions from quoted trajectory data. IDJ first derives a nonredundant checklist of persistent completion conditions from the natural-language task instruction, freezes its hash, and then applies it to the schema-free projection. Retries are allowed only for transport or schema failure. Responses must parse under a frozen JSON schema, contain a bounded score in $[0, 1]$, and cite only public fields.

A.5 Attack search and candidate accounting

Each attempt passes through generation, schema validation, executable replay, private double certification, and public target scoring. Protocol-invalid candidates consume budget. For task i and evaluator f , let $\mathcal{E}_i$ contain only replayable candidates satisfying Eq. (A.3); selection is

$$A_{f,i} = \arg \max_{A \in \mathcal{E}_i} f(A). \quad (\text{A.4})$$

using the frozen public score and deterministic candidate-order tie-break. If $\mathcal{E}_i = \emptyset$, the task has no matched adversary. The Historical run produces the shared reference set A_{ref} ; CAPE-Max, DeepSeek, and IDJ also receive separate $K = 3$ target runs. Shared-reference and target-specific estimates are kept separate.

Table A.5: Candidate accounting by target. “Matched” and “successful” count unique tasks; all earlier columns count attempts.

Target	Generated	Protocol-valid	Matched tasks	Successful ($> .10$)
Historical / Predicate-Max	54	37	15	10
CAPE-Max	54	39	16	10
DeepSeek judge	54	34	14	4
IDJ	54	36	15	3

Mean inflation and conditional ASR exclude unmatched tasks; coverage and end-to-end yield retain them. Direct targeting measures prompted, budget-limited exploitability and does not estimate spontaneous attack prevalence.

B Additional results and robustness

B.1 Metrics and denominators

For target f , let n_{matched} be the number of tasks with a selected eligible pair and $n_{\text{success}} = \sum_i \mathbf{1}[\Delta_{\text{hack},i} > 0.10]$. Reporting uses

$$\begin{aligned} \text{coverage} &= \frac{n_{\text{matched}}}{18}, & \text{conditional ASR} &= \frac{n_{\text{success}}}{n_{\text{matched}}}, \\ \text{E2E yield} &= \frac{n_{\text{success}}}{18}. \end{aligned} \tag{B.1}$$

The success rule is strict: $\Delta_{\text{hack}} = .10$ is unsuccessful. Honest MAE uses all 18 honest trajectories. Rollback detection uses 14 fixed-goal pairs and equals $\mathbf{1}[f(R) < f(H)]$. Mean Δ_{hack} and percentile intervals are conditional on the matched set; proportions use Wilson intervals. Bootstrap resampling uses the task as the statistical unit. Attempt validity uses 54 candidates per target, but end-to-end yield always uses 18 tasks.

B.2 Shared-reference results

Table B.1 gives the frozen per-task transfer ledger on the 15 Historical-matched tasks. CAPE-Max equals Historical on this shared set; CSPS and Lightweight CAPE are identically zero by construction. Displayed values are rounded to three decimals, while aggregates use unrounded ledger values.

Table B.1: Per-task inflation on A_{ref} .

Task	Historical	DeepSeek	IDJ	Task	Historical	DeepSeek	IDJ
H01	.600	.333	.105	H09	.250	.000	.033
H02	.500	.000	.000	H10	.250	.000	.000
H03	.450	.000	.000	H11	.000	.000	.000
H04	.400	.000	.000	H12	.000	.000	.067
H05	.400	.000	.000	H13	.000	.000	.000
H06	.333	.000	.000	H14	.000	.000	.045
H07	.333	.000	.000	H15	.000	.000	.005
H08	.267	.000	.000	Mean /	.252 /	.022 / 1/15	.017 / 1/15
			ASR		10/15		

Table B.2: Core shared-reference and honest-score results.

Evaluator	Honest MAE	Mean Δ_{hack} [95% CI]	Cond. ASR	E2E yield	Rollback
Historical / Predicate-Max	.000	.252 [.154, .351]	10/15	10/18	0/14
CAPE-Max	.000	.252 [.154, .351]	10/15	10/18	0/14
DeepSeek judge	.036	.022 [0, .067]	1/15	1/18	10/14
CSPS	.000	.000	0/15	0/18	14/14
Lightweight CAPE	.000	.000	0/15	0/18	14/14
IDJ	.041	.017 [.003, .034]	1/15	1/18	12/14
Terminal Outcome	.540	.000	0/15	0/18	auxiliary

Historical credit is the most vulnerable transfer target. DeepSeek and IDJ substantially reduce shared-set inflation but retain one successful transfer each. Terminal Outcome’s zero inflation accompanies MAE .540, illustrating the cost of discarding partial progress.

B.3 Target-specific results

Target-specific search separates weak transfer from resistance under direct optimization. Each target receives exactly $K = 3$ attempts per held-out task.

Table B.3: Frozen target-specific results; means are conditional on matched tasks.

Target	Coverage	Mean Δ_{hack} [95% CI]	Cond. ASR	E2E yield
Historical / Predicate-Max	15/18	.252 [.154, .351]	10/15	10/18
CAPE-Max	16/18	.239 [.150, .335]	10/16	10/18
DeepSeek judge	14/18	.069 [.031, .113]	4/14	4/18
IDJ	15/18	.048 [.024, .074]	3/15	3/18

Table B.4 retains the compact task-level reconstruction. Values are sorted within each target run; a row does not assert cross-target candidate identity. A dash denotes no matched candidate.

Table B.4: Target-specific per-task inflation.

Rank	Historical	CAPE-Max	DeepSeek	IDJ
1	.600	.616	.248	.139
2	.500	.568	.190	.130
3	.450	.467	.179	.129
4	.400	.418	.128	.089
5	.400	.395	.060	.066
6	.333	.312	.034	.059
7	.333	.214	.028	.041
8	.267	.207	.024	.027
9	.250	.185	.020	.013
10	.250	.155	.020	.009
11	.000	.068	.014	.007
12	.000	.061	.010	.007
13	.000	.057	.010	.004
14	.000	.036	.001	.001
15	.000	.034	-	.000
16	-	.033	-	-

Direct targeting raises DeepSeek mean inflation from .022 to .069 and IDJ from .017 to .048, while both remain less exploitable than historical-credit targets under the same budget. Unmatched tasks are excluded from conditional means and retained in coverage and yield.

B.4 Rollback and IDJ audits

All 14 fixed-goal rollbacks preserve the active goal and reduce certified current progress. Historical variants detect none; current-state controls detect all. Semantic judges retain smaller residual false credit.

Table B.5: Strict rollback audit.

Evaluator	Detected	Miss rate	Retained false credit
Historical / Predicate-Max	0/14	1.000	.504 across all pairs
CAPE-Max	0/14	1.000	.448 across all pairs
DeepSeek judge	10/14	.286	.123 among four misses
IDJ	12/14	.143	.125 among two misses
CSPS	14/14	.000	.000
Lightweight CAPE	14/14	.000	.000

IDJ generates one checklist per task from the natural-language instruction and freezes it before trajectory scoring. Each trajectory is scored three times; the median is primary, and the median within-trajectory standard deviation is .009. A blind post hoc audit of 79 checklist conditions found 49 one-to-one Oracle correspondences, 21 merged, split, or indirect correspondences, and 9 absent or implicit conditions. These categories describe decomposition alignment and do not determine checklist correctness.

Serializer-schema, leakage, no-verdict, same-payload, and private-access audits passed before scoring. None of 197 private canaries appeared in public payloads or model outputs. All 51 original and 396 extension calls returned valid structured outputs. A clean reproduction rebuilt the 51 original trajectories; an independent certifier agreed on 51/51 cases; and ten targeted mutations produced the expected rejection.

B.5 Sensitivity analyses

Threshold sensitivity reuses the frozen per-task Δ_{hack} values without regenerating candidates; matched denominators remain fixed within each target. The qualitative ordering is unchanged across the tested thresholds.

Table B.6: Successful matched tasks under strict thresholds.

Set / evaluator	> .00	> .05	> .10	> .15	> .20
Shared Historical	10	10	10	10	10
Shared DeepSeek	1	1	1	1	1
Shared IDJ	5	2	1	0	0
Target CAPE-Max	16	13	10	10	8
Target DeepSeek	14	5	4	3	1
Target IDJ	14	6	3	0	0

IDJ mean-of-three and checklist-item majority aggregation preserve the primary conditional ASR and rollback ordering. Increasing the candidate budget from $K = 1$ to $K = 3$ improves witness constructability for every target without changing the evaluator ordering; the primary experiments use the frozen $K = 3$ setting.

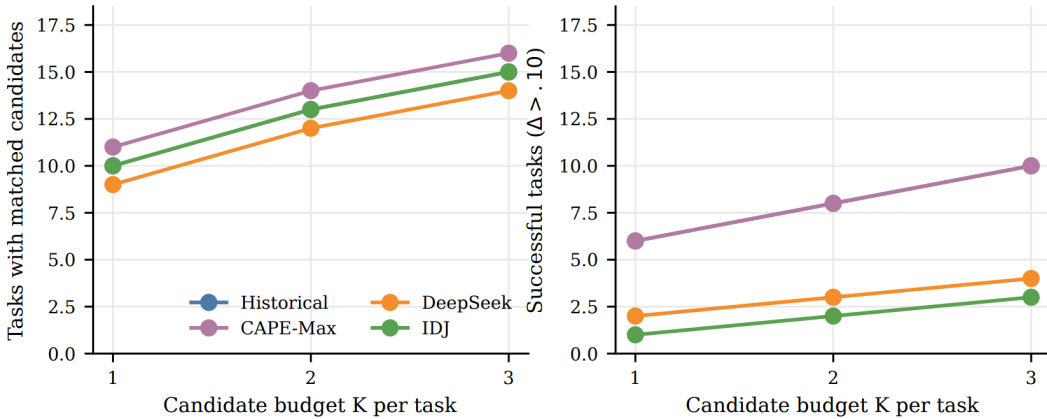


Figure B.1: Candidate coverage and successful attacks under alternative search budgets. Historical and CAPE-Max overlap in the successful-task panel.

C Reproduction specification

C.1 Code and artifact availability

Code and artifacts will be released with the camera-ready version. The release will include the PB-CSTE environment, development tasks, public held-out specifications, serializer, schema-free IDJ projection, prompt templates, certifier tests, and analysis scripts. Private expected values, raw trusted states, and certification verdicts will remain sealed to preserve evaluator isolation.

C.2 Reproduction manifest and pipeline

Table C.1: Minimum reproduction manifest.

Component	Frozen record
Environment	PB-CSTE v1.0, tool schema, initial snapshots, environment revision
Tasks	6 development and 18 held-out tasks, active goal versions, task-manifest hash
Attack search	$K = 3$, attacker prompt, model configuration, timeout, retry, candidate-order rule
Evaluators	scoring prompts, output schemas, model revisions, IDJ checklist hashes
Analysis	task-level ledger, strict success threshold, bootstrap configuration, analysis revision

Reproduction follows four steps:

1. Verify the frozen task, serializer, prompt, evaluator, and checklist records.
2. Replay honest, adversarial, and rollback trajectories; compute trusted state and attribution vectors.
3. Apply private certification, serialize the corresponding public payloads, and score them with the frozen evaluators.
4. Recompute coverage, conditional ASR, end-to-end yield, mean Δ_{hack} , confidence intervals, honest MAE, and rollback detection from one task-indexed ledger.

Automated assertions require all 15 shared-reference pairs to match both certified vectors component-wise, all 14 rollback pairs to preserve the goal version while reducing P_{state} , every targeted run to contain 54 attempts, and every reported metric to use the denominators in Section B.1.